torch.autograd.function.FunctionCtx.ma

https://pytorch.org/docs/stable/generated/torch.autograd.function.FunctionCt

Description:

Mark outputs as non-differentiable. This should be called at most once, in either the setup_context() or forward() methods, and all arguments should be tensor outputs. This will mark outputs as not requiring gradients, increasing the efficiency of backward computation. You still need to accept a gradient for each output in backward(), but it's always going to be a zero tensor with the same shape as the shape of a corresponding output.

Function Signature

```
FunctionCtx.mark_non_differentiable(*args)[source]#

This is used e.g. for indices returned from a sort. See
example::
```

Code Examples

```
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         sorted, idx = x.sort()
>>>         ctx.mark_non_differentiable(idx)
>>>         ctx.save_for_backward(x, idx)
>>>         return sorted, idx
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):  # still need to accept g2
>>>         x, idx = ctx.saved_tensors
>>>         grad_input = torch.zeros_like(x)
>>>         grad_input.index_add_(0, idx, g1)
>>>         return grad_input
```

Detailed Documentation

Documentation

Mark outputs as non-differentiable.

This should be called at most once, in either the setup_context() or forward() methods, and all arguments should be tensor outputs.

This will mark outputs as not requiring gradients, increasing the efficiency of backward computation. You still need to accept a gradient for each output in backward(), but it’s always going to be a zero tensor with the same shape as the shape of a corresponding output.

Mark outputs as non-differentiable.

This should be called at most once, in either the setup_context() or forward() methods, and all arguments should be tensor outputs.

This will mark outputs as not requiring gradients, increasing the efficiency of backward computation. You still need to accept a gradient for each output in backward(), but it’s always going to be a zero tensor with the same shape as the shape of a corresponding output.